ECSE 526
Assignment #3: Reinforcement Learning
Eric Xia
260962364

Generalization:
Function approximation uses any sort of representation for the Q-function other than a lookup table. This strategy uses an evaluation function, represented by a weighted linear function, where the parameter vector theta contains weights that are multiplied by their corresponding features in the feature set $\hat{U}_\theta(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \cdots + \theta_n f_n(s)$.
The set of features contains specialized features that describe the important information during a single state of the game. The state action pairs are specialized by the new version of the update equation of Q-learning $\theta_i \leftarrow \theta_i + \alpha [R(s) + \gamma \max_{a'} \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a)] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$,
which updates the weights at each iteration of the Q-learning agent.
Q(s,a) is defined as the dot product between the vector of weights given the action command and the features at a single state, where the agent is conducting Q-learning, while max Q(s',a') would generate the largest Q-value of all actions given the feature values at a single state. Thus, the specialized features given by the state directly affect the Q-value calculated, which determines the action to be taken, characterizing the state-action pairs.

Similar to the function approximation approach to generalization, the distance-metric approach uses the important information during specific states of the game, multiplied by the corresponding weights (representing the importance of the information given by the corresponding feature). The distance between this state and a future state is calculated using the Euclidean Distance formula using all the information given from a single state.

```
distance = (self.weights['pos_x'] * (pos_x1 - pos_x2) ** 2 +
            self.weights['pos_y'] * (pos_y1 - pos_y2) ** 2 +
            self.weights['prox'] * (prox1 - prox2) ** 2 +
            self.weights['direction'] * (dir1 - dir2) ** 2 +
            self.weights['dots_eaten'] * (dots_eaten1 - dots_eaten2) ** 2 +
            self.weights['stuck'] * (stuck1 - stuck2) ** 2
)
return np.sqrt(distance)
```

The state at each decision point is extracted using the get_state function, which utilizes the RAM mappings given in the ALE documentation linked in the assignment specifications. The information chosen is the player location, the ghost positions, the direction of the game agent, the dots eaten, the locations of the ghosts, and a variable that indicates whether the agent is stuck in a location (to encourage exploration of different moves when the agent is stuck).

The resulting graphs of the 3 generalization implementations of the Ms.Pacman game agent vs the non-generalized implementation are located in Appendix 1. The implementations include a

function approximation agent, a case-based distance metric agent, and an action approximation agent.

In this project, the non-generalized Q-learning agent achieved higher rewards than the generalized agents implemented (function approximation, distance metric between states, and action approximation). This result shows potential trade-offs between the size of the state spaces and the performance of active learning agents. The generalized agents initially were implemented by taking all information within the RAM memory to value the states. However, this resulted in an overly complex environment, severely reducing computational performance. Meanwhile, without generalization, the state representation of the non-generalized agent was able to intake more information from the RAM addresses, thus allowing the agent to train in accordance to the state space of the Ms.Pacman game environment. This can be seen when running the trained agents and viewing the display. The non-generalized agent has more fluid movement and does not get stuck running into walls nearly as much as the generalized agents. In hindsight, I would have attempted to improve the information features of the function approximation algorithm to better generalize the information received from the state space.

Exploration:
The exploration function primarily used in the game agent implementations is the optimistic prior approach described in section 21.3.1 of the AIMA textbook. The expression is given by:

$$f(u,n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{otherwise} \end{cases}$$

where R+ was set to be the optimistic estimate of the best possible reward in any state: 3000, Ne was set as 5.0 (as an Ne that is too high requires an abundance of episodes before this is fully explored, and an Ne that is too low causes minimal exploration), and u is the Q-value at a given state.
The agent would choose actions similarly to the pseudocode of the exploratory Q-learning agent, adjusted for generalization:
By the pseudocode provided in section 21.3.2 of the exploratory Q-learning agent, after the Q-table (or weight vectors in function approximation) is updated, the agent would attempt to find the best action by maximizing the exploration function. This is conducted by calculating argmax_a'f(Q[s',a'], Nsa[s',a']), where the agent would collect a list of all possible Q-values with the current state by testing each possible action, and would input each Q and Nsa value into the exploration function, where the action that results in the highest exploration value is chosen.

Another approach to exploration was a random epsilon exploration function, where exploration is randomized at a high (initially 90) percentage. When this occurs, the R+ value is returned and otherwise, the Q-value is returned. The epsilon variable would further decay to reduce exploration as more episodes are conducted to exploit the data that was collected.

The resulting graphs of the 2 exploration implementations of the Ms.Pacman game agent vs the implementation without exploration are located in Appendix 2. The implementations include an optimistic prior approach and an epsilon-random approach, utilizing the function approximation agent.

The optimistic prior approach was chosen as this encourages the agent to attempt every reachable state-action pair, assigning a high initial value to unvisited actions. This encourages the agent to explore the environment until each action has been tried at least Ne times, ensuring that learning is based on several representative sets of experiences to combat outliers and overfitting.
As can be seen in the resulting graphs [Appendix 2], the optimistic prior approach to exploration leads to the greatest amount of learning during training, with the agent averaging the highest reward by the end of learning. When the agents were ran with a different random seed, the optimistic prior approach also achieved the best performance. Thus, this form of exploration resulted in the agent developing better learning in various situations, while the other agents may have learnt unoptimal strategies due to the less effective exploration.

Agent Performance:
The resulting graphs of the agent performance of the Ms.Pacman game agent tested with different seeds after training are located in Appendix 2. The implementations include an optimistic prior approach to the function approximation agent, where 1000 episodes of training are conducted, and the Q-table saved from the training is used on 5 different random seeds to test for generalization across varied states.

The agent's performance was measured by training the Function Approximation agent for 1000 episodes. Then, the agent was set to run on 5 random seeds without learning for 100 episodes, as shown in this graph. As the agent is no longer learning at this point, the reward values across the curve of each seed stay fairly compact, aside from a couple of outliers (such as episode ~ 2 for seed 0). You can further see that the curve of each seed outputs fairly similar reward values, all concentrated around a reward of 300, as shown by the mean curve. By utilizing the learning from the previous training and action approximation generalization, the performance of the agent is fairly consistent throughout differing seeds, where the behaviour of the environment is unknown.

Strategies Used:

1. Ghost Avoidance:
    - When accessing the RAM memory to return the current state, the distance from the nearest ghost is returned.
    - When a ghost is close to the agent, the value returned would be low. In Function approximation, as the min_distance of the feature value would be low in this case, the agent would rather be further away from the ghosts to calculate a higher Q-value.
2. Stuck Detection:
    - When accessing the RAM memory to return the current state, the agent checks the current and previous locations and actions of the player. Should these all be the same, then the stuck indicator would be set.
    - When the stuck indicator is set, there is a bonus penalty given to the agent, reducing the reward the agent sees (not the total reward of the game). This encourages the agent to leave the state when stuck.

3. Collecting Dots:
    - By using the normalized dots eaten count as a feature, the agent would be encouraged to eat more dots to calculate a higher Q-value, and thus a higher reward/
4. Fruit Proximity:
    - The feature relating to the fruit location receives a higher value when the fruit is closer to the agent. Thus, the agent would be encouraged to move closer to the fruit to maximize the Q-value
5. Score Tracking:
    - The feature relating to the score correlates actions with the total score, thus encouraging actions that increase the future score as much as possible (while keeping track of other features).

Appendix:
AI-Generated Code:
While coding, GitHub Copilot was running in my IDE, Visual Studio Code. There were occasional code suggestions and line completions that were used throughout the program

Prompt:
Can you speedup the computational performance of my game agent?
Input:

```
    def q_learning_agent(self, percept):
        s_prime, r_prime = percept

        # if terminal state, then Q[s, None] <- r'
        if self.s is not None and self.is_terminal(self.s):
            self.Q[self.s][None] = r_prime

        # if s is not None increment Nsa[s,a]
        if self.s is not None:
            self.Nsa[self.s][self.a] += 1

            if not self.is_terminal(s_prime):
                # Compute max_a' Q[s',a'] with generalization
                max_q = float('-inf')
                for a in range(self.num_actions):
                    q_value = self.generalized_qval(s_prime, a,
self.Nsa[s_prime][a])
                    if q_value > max_q:
                        max_q = q_value
            else:
                max_q = 0.0

            # Q[s,a] <- Q[s,a] + alpha(Nsa[s,a]) × (r + gamma × max_a'
Q[s',a'] - Q[s,a])
            q_current = self.Q[self.s][self.a]
            alpha = self.get_alpha(self.Nsa[self.s][self.a])
            self.Q[self.s][self.a] = q_current + alpha * (self.r +
self.gamma * max_q - q_current)

        if not self.is_terminal(s_prime):
            best_action = None
            best_value = float('-inf')
```

```
            # find argmax_a' f(Q[s',a'], Nsa[s',a'])
            for a in range(self.num_actions):
                q_value = self.generalized_qval(s_prime, a,
self.Nsa[s_prime][a])
                exploration = self.exploration_function(q_value,
self.Nsa[s_prime][a])

                if exploration > best_value:
                    best_value = exploration
                    best_action = a

            self.s = s_prime
            self.a = best_action
            self.r = r_prime
            return best_action
        else:
            # Terminal state
            self.s = s_prime
            self.a = None
            self.r = r_prime
            return None
```

Output: Computational optimizations:

```
def get_cached_similar_states(self, state):
        """Cache similar states for frequently visited states"""
        state_key = str(state)
        total_visits = sum(self.Nsa[state][a] for a in
range(self.num_actions))

        # Only cache frequently visited states
        if total_visits >= self.cache_hit_threshold:
            if state_key not in self.similar_states_cache:
                self.similar_states_cache[state_key] =
self.similar_states(state)
            return self.similar_states_cache[state_key]
        else:
            # For rarely visited states, compute on demand (no cache)
            return self.similar_states(state)
```

```
def q_learning_agent(self, percept, learning=True):
        s_prime, r_prime = percept

        # Only update Q-table and Nsa if learning is enabled
        if learning:
            # if terminal state, then Q[s, None] <- r'
            if self.s is not None and self.is_terminal(self.s):
                self.Q[self.s][None] = r_prime

        # if s is not None increment Nsa[s,a]
        if self.s is not None:
            self.Nsa[self.s][self.a] += 1

            if not self.is_terminal(s_prime):
                # OPTIMIZATION: Compute all generalized Q-values once
                generalized_q_values = []
                for a in range(self.num_actions):
                    q_value = self.generalized_qval(s_prime, a,
self.Nsa[s_prime][a])
                    generalized_q_values.append(q_value)

                max_q = max(generalized_q_values)
            else:
                max_q = 0.0
                generalized_q_values = None

            # Q-learning update
            q_current = self.Q[self.s][self.a]
            self.Q[self.s][self.a] = q_current + self.alpha * (self.r +
self.gamma * max_q - q_current)

        if not self.is_terminal(s_prime):
            best_action = None
            best_value = float('-inf')

            # OPTIMIZATION: Reuse computed Q-values if available
            if 'generalized_q_values' in locals() and generalized_q_values
is not None:
                q_values = generalized_q_values
            else:
```

```
                # Compute once for action selection only
                q_values = []
                for a in range(self.num_actions):
                    q_value = self.generalized_qval(s_prime, a,
self.Nsa[s_prime][a])
                    q_values.append(q_value)
```

Prompt:
Can you create a graphs.py file that runs each of my generalized agents, the tabular agent with a target value of 400, running 1000 episodes for each agent. Create a bar graph showing the episode in which the agent reaches the target value. After learning, run 100 episodes on a different seed and create a graph with the final performance of each agent.
Input: a3action.py, a3distance.py, a3func.py, a3nongen.py
Output:
graphs.py file (end product is adjusted from the AI output)

Prompt:
Can you create a agentperformance.py file that trains a3action.py with 1000 episodes, then with 5 random seeds, run a3action.py without learning with 100 episodes to see the results of generalization. Then create a graph using this data.
Input: a3action.py
Output:
agentperformance.py (end product is adjusted from the AI output)

Prompt:
My agent keeps getting stuck on walls. How do I create a variable in the state that indicates if the agent is stuck?

```
def get_state(self):
        ram = self.ale.getRAM()

        # RAM locations taken from the ALE documentation and Ms. Pacman
specifics
        player_x = ram[10] // 16
        player_y = ram[16] // 16
        ghost_positions = [
            (ram[6] // 16, ram[12] // 16),    # Sue
            (ram[7] // 16, ram[13] // 16),    # Inky
            (ram[8] // 16, ram[14] // 16),    # Pinky
            (ram[9] // 16, ram[15] // 16)     # Blinky
        ]

        # Calculate proximity to nearest ghost
```

```
        ghost_distances = []
        for gx, gy in ghost_positions:
            if gx > 0 and gy > 0:
                dist = abs(player_x - gx) + abs(player_y - gy)
                ghost_distances.append(dist)

        min_distance = 0
        # Check nearest ghost distance and proximity
        if ghost_distances:
            min_distance = min(ghost_distances)

        direction = ram[56] // 64
        dots_eaten_count = ram[119] / 151.0
        fruit_x = ram[11] // 16
        fruit_y = ram[17] // 16
        player_score = ram[120] / 1000

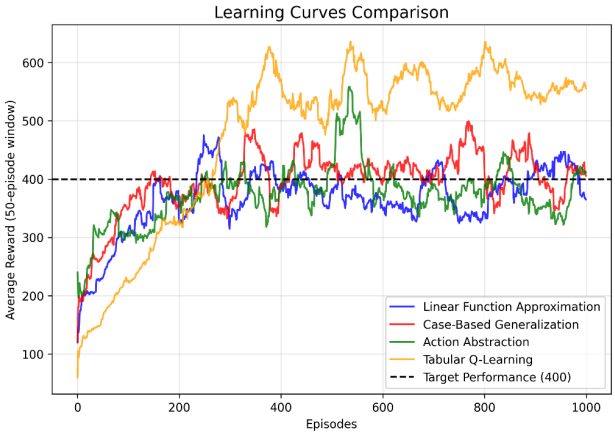
        return (player_x, player_y, min_distance, direction,
dots_eaten_count, fruit_x, fruit_y, player_score)
```

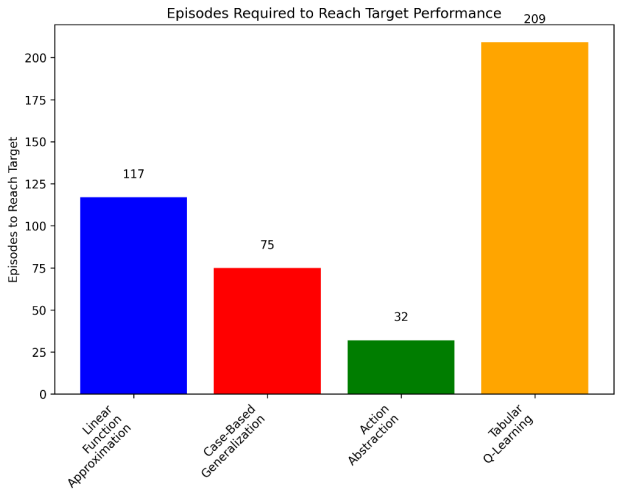
Output:

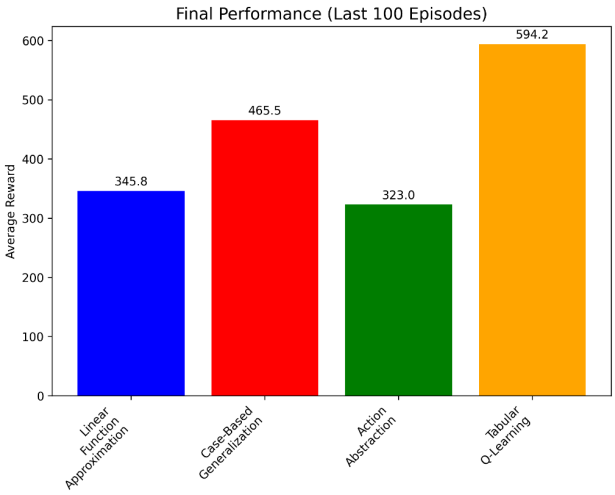
```
        prev_player_x = getattr(self, 'prev_player_x', player_x)
        prev_player_y = getattr(self, 'prev_player_y', player_y)
        # Check if agent is stuck
        is_stuck = (player_x == prev_player_x and player_y ==
prev_player_y and hasattr(self, 'prev_action') and self.prev_action is not
None)
        self.prev_player_x = player_x
        self.prev_player_y = player_y
        if is_stuck:
            stuck_indicator = 1
        else:
            stuck_indicator = 0
```

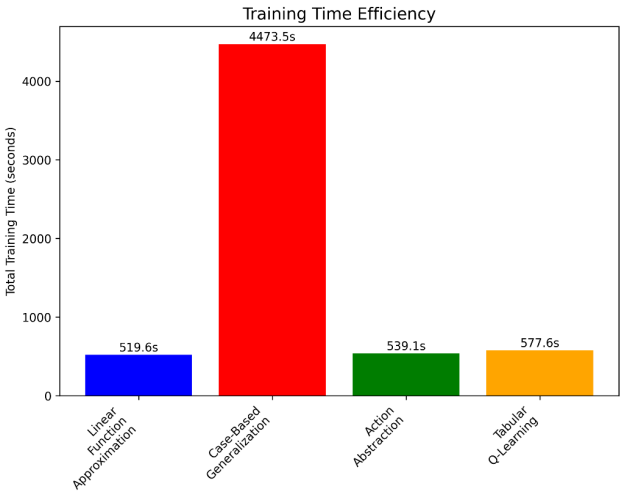
Graphs:

1. Generalization graphs

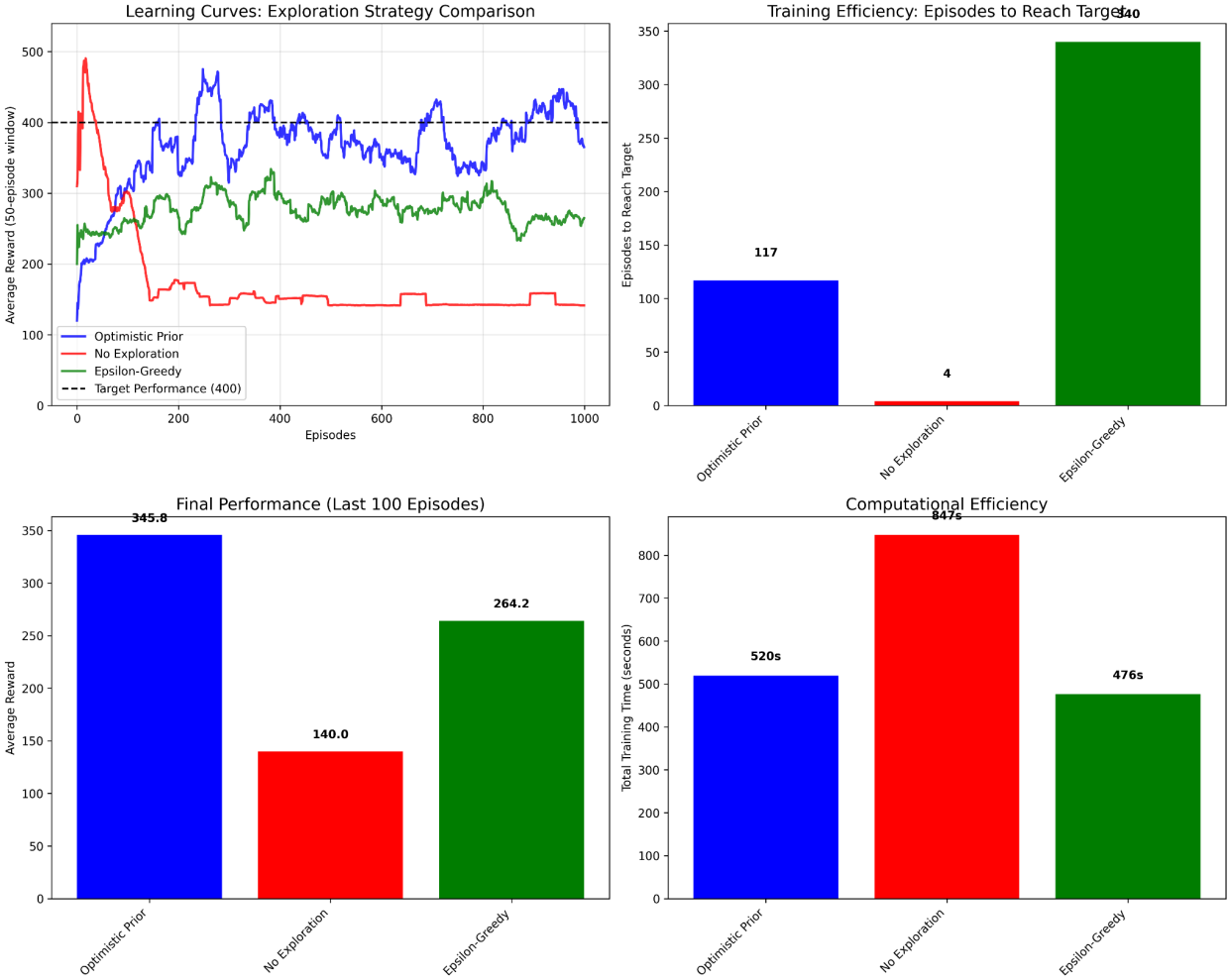








2. Exploration graphs



3. Agent Performance Generalization Across Seeds graph

